Saiyed Shizain | CU24250233 | Roll: 48
BTech CSE (A) | 3rd Yr - 5th Sem | 2024-2028

Full-Stack Web Development — BTCS303P

LAB SHEET — 5: React Forms: Validation, Strength Meter, Wizard

Task 5.1 — Login form with regex validation

Aim. Controlled email/password inputs with live regex validation and error/success badges.

Theory. Controlled inputs keep React state as the source of truth; touched flags prevent errors flashing before interaction.

```
import { useState } from "react";

// Task 5.1 — Controlled login form with regex email/password validation
// and interactive error badges rendered on invalid patterns.
const EMAIL_PATTERN = /^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$/;
const PASSWORD_PATTERN =
  /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@$!%*?&#^()_\-+=]).{8,}$/;

function validateEmail(value) {
  if (!value) return "Email is required.";
  if (!EMAIL_PATTERN.test(value))
    return "Enter a valid email address (e.g. name@domain.com).";
  return "";
}

function validatePassword(value) {
  if (!value) return "Password is required.";
  if (value.length < 8) return "Password must be at least 8 characters.";
  if (!PASSWORD_PATTERN.test(value))
    return "Must include uppercase, lowercase, number & special character.";
  return "";
}

export default function LoginForm() {
  const [formData, setFormData] = useState({ email: "", password: "" });
  const [errors, setErrors] = useState({ email: "", password: "" });
  const [touched, setTouched] = useState({ email: false, password: false });
  const [submitted, setSubmitted] = useState(false);

  const handleChange = (e) => {
    const { name, value } = e.target;
    setFormData((prev) => ({ ...prev, [name]: value }));
    const errorMsg =
      name === "email" ? validateEmail(value) : validatePassword(value);
    setErrors((prev) => ({ ...prev, [name]: errorMsg }));
  };

  const handleBlur = (e) => {
    const { name } = e.target;
    setTouched((prev) => ({ ...prev, [name]: true }));
  };

  const handleSubmit = (e) => {
    e.preventDefault();
    const emailError = validateEmail(formData.email);
    const passwordError = validatePassword(formData.password);
    setErrors({ email: emailError, password: passwordError });
    setTouched({ email: true, password: true });
    if (!emailError && !passwordError) {
      setSubmitted(true);
    }
  };
```

```
  return (
    <div className="card">
      <h2 className="card-title">Sign in to your account</h2>
      <p className="card-subtitle">Task 5.1 — Controlled Login Form with Regex Validatio
n</p>
      <form onSubmit={handleSubmit} noValidate>
        <div className="field-group">
          <label className="field-label" htmlFor="email">Email address</label>
... [51 lines truncated for print]
```

```
$ npm run build
✓ 33 modules transformed.
$ curl served index: 200
Login tab renders Sign-in card (bundle verified; typing behavior inspected, no browser
available)
```

Conclusion. Validation logic and badges implemented per reference. — Saiyed Shizain

---

Task 5.2 — Reusable password-strength sub-component

Aim. Score-based strength bar plus rule checklist, reusable via props in any parent.

Theory. Each satisfied rule adds a point (bonus for length ≥ 12); useMemo avoids recomputation; the child owns no password state.

```
import { useMemo } from "react";

// Task 5.2 — Reusable password strength checker: maps input variations to
// a dynamic progress bar. Receives `password` via props for reuse anywhere.
const STRENGTH_LEVELS = [
  { label: "Very Weak", color: "#ff5b6e", width: "20%" },
  { label: "Weak", color: "#ff8c42", width: "40%" },
  { label: "Fair", color: "#f5b642", width: "60%" },
  { label: "Good", color: "#4f8bff", width: "80%" },
  { label: "Strong", color: "#2ecf7d", width: "100%" },
];

function scorePassword(password) {
  if (!password) return 0;
  let score = 0;
  const rules = {
    length: password.length >= 8,
    lowercase: /[a-z]/.test(password),
    uppercase: /[A-Z]/.test(password),
    number: /\d/.test(password),
    special: /[@$!%*?&#^()_\-+=]/.test(password),
  };
  Object.values(rules).forEach((met) => met && score++);
  if (password.length >= 12) score++;
  return { score: Math.min(score, 5), rules };
}

export default function PasswordStrengthChecker({ password, showRules = true }) {
  const { score, rules } = useMemo(() => {
    const result = scorePassword(password);
    return result === 0 ? { score: 0, rules: {} } : result;
  }, [password]);
  const levelIndex = Math.max(score - 1, 0);
  const level = password ? STRENGTH_LEVELS[levelIndex] : null;
  return (
    <div className="strength-checker">
      <div className="strength-bar-track">
        <div
          className="strength-bar-fill"
          style={{
            width: level ? level.width : "0%",
```

```
            background: level ? level.color : "transparent",
          }}
        />
      </div>
      <div className="strength-label-row">
        <span className="strength-label" style={{ color: level ? level.color : "#6b7280"
 }}>
          {level ? level.label : "Enter a password"}
        </span>
        <span>{password ? `${password.length} chars` : ""}</span>
      </div>
      {showRules && password && (
        <div className="strength-rules">
          <div className={`strength-rule ${rules.length ? "met" : ""}`}>
            <span className="rule-dot" /> Minimum 8 characters
          </div>
          <div className={`strength-rule ${rules.uppercase ? "met" : ""}`}>
            <span className="rule-dot" /> Uppercase letter
          </div>
          <div className={`strength-rule ${rules.lowercase ? "met" : ""}`}>
... [14 lines truncated for print]
```

```
import { useState } from "react";
import PasswordStrengthChecker from "./PasswordStrengthChecker.jsx";

// Parent owning the password state, passed down to the reusable checker.
export default function PasswordStrengthDemo() {
  const [password, setPassword] = useState("");
  return (
    <div className="card">
      <h2 className="card-title">Create a new password</h2>
      <p className="card-subtitle">Task 5.2 — Reusable Password Strength Sub-Component</
p>
      <div className="field-group">
        <label className="field-label" htmlFor="new-password">New password</label>
        <input
          id="new-password"
          type="password"
          className="field-input"
          placeholder="Type to test strength..."
          value={password}
          onChange={(e) => setPassword(e.target.value)}
        />
      </div>
      <PasswordStrengthChecker password={password} showRules />
      <button className="primary-btn" style={{ marginTop: 24 }}>
        Set Password
      </button>
    </div>
  );
}
```

Conclusion. Parent owns state, child renders purely from props. — Saiyed Shizain

---

Task 5.3 — Multi-step onboarding wizard

Aim. Three-step wizard with all state in the parent and controlled step children.

Theory. updateFormData merges partial updates; step index drives the stepper, Back/Next clamping and final submit.

```
import { useState } from "react";
import StepAccount from "./steps/StepAccount.jsx";
import StepPersonalInfo from "./steps/StepPersonalInfo.jsx";
import StepFinish from "./steps/StepFinish.jsx";
```

```
// Task 5.3 — Multi-step onboarding wizard. The parent owns ALL wizard state;
// children are fully controlled via data + onChange props.
const STEP_LABELS = ["Account", "Personal Info", "Finish"];

const initialFormState = {
  username: "",
  email: "",
  password: "",
  fullName: "",
  course: "",
  city: "",
};

export default function OnboardingWizard() {
  const [step, setStep] = useState(0);
  const [formData, setFormData] = useState(initialFormState);
  const [submitted, setSubmitted] = useState(false);

  const updateFormData = (partial) => {
    setFormData((prev) => ({ ...prev, ...partial }));
  };
  const goNext = () => setStep((s) => Math.min(s + 1, STEP_LABELS.length - 1));
  const goBack = () => setStep((s) => Math.max(s - 1, 0));
  const handleFinalSubmit = () => {
    setSubmitted(true);
  };

  const renderStep = () => {
    switch (step) {
      case 0:
        return <StepAccount data={formData} onChange={updateFormData} />;
      case 1:
        return <StepPersonalInfo data={formData} onChange={updateFormData} />;
      case 2:
        return <StepFinish data={formData} submitted={submitted} />;
      default:
        return null;
    }
  };

  return (
    <div className="card">
      <h2 className="card-title">Onboarding Wizard</h2>
      <p className="card-subtitle">Task 5.3 — Parent-owned multi-step state</p>
      <ol className="stepper">
        {STEP_LABELS.map((label, i) => (
          <li key={label} className={i === step ? "step current" : i < step ? "step done
" : "step"}>
            {i + 1}. {label}
          </li>
        ))}
      </ol>
      {renderStep()}
      <div className="wizard-nav">
        {step > 0 && (
          <button className="secondary-btn" onClick={goBack}>Back</button>
... [12 lines truncated for print]
```

```
// Step 1 — Account credentials. Controlled child: no local state.
export default function StepAccount({ data, onChange }) {
  const handle = (e) => onChange({ [e.target.name]: e.target.value });
  return (
    <div>
      <div className="field-group">
        <label className="field-label" htmlFor="username">Username</label>
        <input
          id="username"
```

```
          name="username"
          className="field-input"
          placeholder="e.g. saiyed_s"
          value={data.username}
          onChange={handle}
        />
      </div>
      <div className="field-group">
        <label className="field-label" htmlFor="onboard-email">Email address</label>
        <input
          id="onboard-email"
          name="email"
          className="field-input"
          placeholder="you@example.com"
          value={data.email}
          onChange={handle}
        />
      </div>
      <div className="field-group">
        <label className="field-label" htmlFor="onboard-password">Password</label>
        <input
          id="onboard-password"
          name="password"
          type="password"
          className="field-input"
          placeholder="Create a password"
          value={data.password}
          onChange={handle}
        />
      </div>
    </div>
  );
}
```

```
// Step 2 — Personal information. Controlled child like Step 1.
export default function StepPersonalInfo({ data, onChange }) {
  const handle = (e) => onChange({ [e.target.name]: e.target.value });
  return (
    <div>
      <div className="field-group">
        <label className="field-label" htmlFor="fullName">Full name</label>
        <input
          id="fullName"
          name="fullName"
          className="field-input"
          placeholder="e.g. Saiyed Shizain"
          value={data.fullName}
          onChange={handle}
        />
      </div>
      <div className="field-group">
        <label className="field-label" htmlFor="course">Course / Program</label>
        <input
          id="course"
          name="course"
          className="field-input"
          placeholder="e.g. BTech CSE"
          value={data.course}
          onChange={handle}
        />
      </div>
      <div className="field-group">
        <label className="field-label" htmlFor="city">City</label>
        <input
          id="city"
          name="city"
```

```
          className="field-input"
          placeholder="e.g. New Delhi"
          value={data.city}
          onChange={handle}
        />
      </div>
    </div>
  );
}
```

```
// Step 3 — Review everything, then confirm submission.
export default function StepFinish({ data, submitted }) {
  const rows = [
    ["Username", data.username],
    ["Email", data.email],
    ["Full name", data.fullName],
    ["Course", data.course],
    ["City", data.city],
  ];
  return (
    <div>
      <h3>Review your details</h3>
      <dl className="review-list">
        {rows.map(([label, value]) => (
          <div key={label} className="review-row">
            <dt>{label}</dt>
            <dd>{value || "—"}</dd>
          </div>
        ))}
      </dl>
      {submitted && (
        <p className="footer-note">Onboarding complete — welcome aboard!</p>
      )}
    </div>
  );
}
```

Conclusion. Props-based parent-child communication throughout. — Saiyed Shizain

---

Shell — App, styles, build proof

Aim. Tab switcher shell plus shared card/field/badge styling; prove the production build.

```
import { useState } from "react";
import LoginForm from "./components/LoginForm.jsx";
import PasswordStrengthDemo from "./components/PasswordStrengthDemo.jsx";
import OnboardingWizard from "./wizard/OnboardingWizard.jsx";

// Lab Sheet 5 — Saiyed Shizain: three form demos behind a simple tab switcher.
const TABS = [
  { id: "login", label: "5.1 Login Form" },
  { id: "strength", label: "5.2 Password Strength" },
  { id: "wizard", label: "5.3 Onboarding Wizard" },
];

function App() {
  const [tab, setTab] = useState("login");
  return (
    <div className="app">
      <header className="app-header">
        <h1>React Forms Lab</h1>
        <p>Saiyed Shizain — controlled inputs, validation, parent-child state</p>
        <nav className="tabs">
          {TABS.map((t) => (
            <button
```

```
              key={t.id}
              className={tab === t.id ? "tab active" : "tab"}
              onClick={() => setTab(t.id)}
            >
              {t.label}
            </button>
          ))}
        </nav>
      </header>
      {tab === "login" && <LoginForm />}
      {tab === "strength" && <PasswordStrengthDemo />}
      {tab === "wizard" && <OnboardingWizard />}
    </div>
  );
}

export default App;
```

```
> vite build
✓ 33 modules transformed.
dist/assets/index-FHhPx52N.js 151.13 kB
$ curl -s http://127.0.0.1:8815/ | grep -o '<title>[^<]*</title>'
<title>React Forms Lab — Saiyed Shizain</title>
```